

Maven-Modul einbinden: Dependency vs. Workspace-Modul in IntelliJ

Confluence-Artikel · Entwicklungsnotizen · IntelliJ / Maven

📌 Auf einen Blick

Bei einer reinen Maven-**Dependency** arbeitest du mit dem gebauten Artefakt (JAR aus `.m2`). Beim **Modul im Workspace** arbeitest du mit dem echten Quellcode und dessen `target/classes`. Der Unterschied entscheidet darüber, ob Änderungen sofort wirken – und was du selbst bauen musst, wenn Auto-Build aus ist.

1. Dependency vs. Workspace-Modul

Der Kern: Die reine **Maven-Dependency** liefert dir das *gebaute Artefakt* (JAR), das **Workspace-Modul** den *Quellcode selbst*.

Nur als Maven-Dependency (<dependency> im pom.xml)

IntelliJ zieht das Projekt als fertiges JAR aus deinem lokalen Repository (`~/.m2/repository`) oder einem Remote-Repo.

- Du siehst nur die kompilierte Version – Quellcode nur bei angehängtem sources-JAR, und dann **read-only**.
- Änderungen am anderen Projekt werden erst nach `mvn install` sichtbar (neues JAR im `.m2`).
- Refactoring, *Find Usages* und *Go to Definition* enden an der Projektgrenze – das andere Projekt ist eine Blackbox.
- Im Debugger landest du bestenfalls in angehängtem/dekompiliertem, nicht editierbarem Code.

Als Modul in den Workspace geladen

Du lädst das Projekt (bzw. dessen pom.xml) als zusätzliches Modul in dasselbe IntelliJ-Projekt. Stimmen die Maven-Koordinaten (groupId/artifactId/**version**) mit der Dependency überein, ersetzt IntelliJ die JAR-Abhängigkeit automatisch durch eine **Modul-Abhängigkeit**.

- Echter Quellcode – editierbar, navigierbar, refactorbar über beide Projekte hinweg.
- Änderungen wirken sofort, ohne `mvn install` dazwischen.
- Der Debugger springt in den echten, editierbaren Code.
- IntelliJ kompiliert beide Module gemeinsam gegen die aktuellen Quellen.

✓ Faustregel

Reine Dependency für stabile Fremd-Bibliotheken, die du nur *nutzt*. Workspace-Modul, wenn du parallel an beiden entwickelst.

⚠ Stolperstein

Damit die Ersetzung greift, muss die Version in der Dependency **exakt** zur Modul-Version passen (z. B. `1.0.0-SNAPSHOT`). Sonst zieht IntelliJ doch wieder das JAR aus `.m2` – und du wunderst dich, warum deine Änderungen nicht ankommen.

2. Die Rolle von `target/classes`

`target/classes` ist die **Compiler-Ausgabe**: Hierhin legt Maven die `.class`-Dateien deines Hauptquellcodes (Testklassen parallel in `target/test-classes`). Es ist der Zwischenstand zwischen Quellcode und JAR – beim `mvn package / install` wird genau dieser Inhalt ins JAR gezippt und ins `.m2` kopiert.

	Was liegt auf dem Classpath?	Aktualität
Workspace-Modul	direkt <code>target/classes</code> des Moduls	so aktuell wie der letzte Build
JAR-Dependency	JAR aus <code>.m2</code>	Momentaufnahme vom letzten <code>mvn install</code>

Beim **Modul im Workspace** setzt IntelliJ nicht ein JAR, sondern direkt das Output-Verzeichnis auf den Classpath. Bei Maven-Projekten teilen sich IDE und Maven dabei standardmäßig **dasselbe** `target/classes` (statt `out/`). Änderst du eine Klasse, rekompiliert IntelliJ sie dorthin und beim nächsten Run/Debug ist sie sofort aktiv.

Bei der **JAR-Dependency** ist `target/classes` des anderen Projekts für dich irrelevant – dein Classpath kennt nur das JAR. Die Brücke führt über `target/classes` → JAR → `.m2`, und ohne erneutes `mvn install` kommt nichts bei dir an.

⚠ Stolperstein - veraltete `.class`-Dateien

Weil IDE und Maven sich `target/classes` teilen, können dort alte Stände liegenbleiben (nach Branch-Wechsel, gelöschten Klassen, Wechsel zwischen IntelliJ- und Maven-Build). Abhilfe: `mvn clean` oder in IntelliJ *Rebuild Project*.

3. Braucht der Compiler den Bytecode?

Ja. Zur Kompilierzeit müssen die importierten Klassen dem Compiler als fertiger **Bytecode** (`.class`) auf dem Classpath vorliegen. `javac` löst Typen nicht über fremden Quellcode auf, sondern liest Signaturen (Methoden, Felder, Typen) aus dem Bytecode. Fehlt er, gibt es `cannot find symbol` bzw. `package ... does not exist`.

Normalerweise musst du dich **nicht selbst** darum kümmern, weil das Build-System die Reihenfolge übernimmt:

- **Modul-Abhängigkeit in IntelliJ:** Die IDE kennt den Abhängigkeitsgraphen, baut zuerst das abhängige Modul (füllt dessen `target/classes`) und dann dein Modul dagegen.
- **Maven-Reactor** (beide Module unter einem Parent): Maven sortiert topologisch, baut das abhängige Modul zuerst und greift im selben Lauf direkt auf dessen `target/classes` zu – **kein** `mvn install` nötig.

Kritisch wird es nur bei **isolierter** Kompilierung, wenn die Quelle weder als gebautes Modul noch als installiertes JAR vorliegt (z. B. `mvn compile` nur in deinem Projekt, das andere nie installiert und kein Reactor-Geschwister). Dann bricht der Compiler ab.

Präzisierung

Gefordert ist „Bytecode irgendwo auf dem Classpath“ – beim Modul ist das `target/classes` , bei der JAR-Dependency das JAR. Der Ort wechselt, die Anforderung bleibt.

4. Sonderfall: Auto-Build in IntelliJ ausgeschaltet

Bei sehr großen Projekten schaltet man *Build project automatically* oft ab. Damit fällt genau die Automatik weg, die `target/classes` des abhängigen Moduls von selbst füllt. **Der Bytecode ist dann nur so aktuell wie dein letzter manueller Build.**

- Der Compiler-Classpath zeigt weiter auf `target/classes` des anderen Moduls – ob dort *aktueller* Bytecode liegt, hängt jetzt allein an dir.
- Änderst du eine Klasse im abhängigen Modul und baust nicht neu, kompilierst du gegen den **alten** Stand.
- Wurde die Klasse dort nie kompiliert, schlägt dein Build mit `cannot find symbol` fehl – obwohl der Quellcode sichtbar im Workspace liegt.

Praktikable Wege

- **Gezielt nur das Modul bauen:** `Build → Build Module 'xyz'` statt jedes Mal das ganze Projekt. Hält die Bauzeit bei großen Projekten überschaubar.
- **Maven direkt:** `mvn install` im abhängigen Modul, oder Reactor-Build von der Parent-Ebene:

```
mvn -pl deinModul -am compile
```

`-pl` wählt dein Modul, `-am` („also make“) baut alle Module mit, von denen es abhängt – in korrekter Reihenfolge. Bei ausgeschaltetem Auto-Build oft der verlässlichste Weg, weil Maven die Reihenfolge selbst erzwingt.

- **Kompromiss:** Auto-Build global aus, aber in der *Run/Debug Configuration* einen gezielten Build- oder Maven-Schritt als *Before launch* nur für die wirklich benötigten Module hinterlegen. So baust du nicht ständig alles, hast aber vor dem Start frischen Bytecode für den relevanten Pfad.

Roter Faden

Ohne Auto-Build musst du selbst dafür sorgen, dass der Bytecode des abhängigen Moduls vor deiner Kompilierung existiert und aktuell ist. IntelliJ nimmt dir diese Reihenfolge dann nicht mehr ab.